

Cardano Registry vs Certificate Storage Architecture

Why the Registry Is NOT Used to Store Certificates

A common misunderstanding in Cardano's eUTXO model is assuming that the issuer registry stores all certificate records.

In this architecture, the registry is **not** intended to hold certificate data.

Instead, it acts as a:

- lightweight authorization layer
- governance state
- access-control registry

The actual certificates are stored independently.

1. The Registry UTXO

The registry contract maintains a single state UTXO:

```
Registry UTXO
├─ admin
├─ authorized issuers[]
└─ version
```

Purpose:

- identify which wallets are authorized to issue certificates
- allow admin-controlled issuer management
- provide a canonical on-chain source of issuer authorization

This UTXO is small and changes only when:

- an issuer is added
- an issuer is removed
- admin authority rotates

Therefore, the registry is scalable because:

- updates are infrequent
- the datum remains compact
- only one active registry state exists

The registry does NOT store certificates.

2. Certificate Storage

Each certificate is stored independently.

There are two possible designs.

Option A — Certificate Tokens (Recommended)

Each certificate becomes a native token/NFT minted under a certificate policy.

Example:

```
UTXO
├ 2 ADA
└ CERT_GEM_001
```

Metadata or datum contains:

```
{
  "gemId": "GEM_001",
  "issuer": "GIA",
  "certHash": "...",
  "reportId": "R001",
  "documentCid": "ipfs://..."
}
```

Certificates are distributed across many UTXOs and wallets.

This is highly scalable because:

- certificates are independent
- no central state bottleneck exists
- new certificates do not modify old certificates

- issuance is append-only
-

Option B — Certificate Script UTXOs

Each certificate is stored as a UTXO at a certificate validator address.

Example:

```
Certificate Validator Address
├─ UTXO 1 → CertificateDatum(GEM_001)
├─ UTXO 2 → CertificateDatum(GEM_002)
├─ UTXO 3 → CertificateDatum(GEM_003)
```

This also scales because each certificate exists independently.

However, for most applications, token + metadata approaches are simpler.

3. Why the Registry Remains Scalable

The registry is NOT a list of certificates.

It is only:

```
Who is allowed to issue certificates?
```

Even if the system contains:

- 10 issuers
- 10,000 certificates
- 1 million events

The registry still only stores:

```
authorized issuer PKHs
```

Therefore the registry remains small and efficient.

4. Role of the Minting Policy

The certificate minting policy acts as the authorization gate.

Before allowing certificate minting, it:

1. references the registry UTXO
2. reads the authorized issuer list
3. checks transaction signers
4. allows minting only for approved issuers

Conceptually:

```
Registry UTXO
  ↓
Certificate Minting Policy
  ↓
Certificate Token Minted
```

The minting policy does not store certificates.

It only validates whether minting is permitted.

5. Final Architecture

```
Issuer Registry Validator
  ↓ stores
Registry UTXO
  ↓ authorizes
Certificate Minting Policy
  ↓ allows
Certificate Token Minting
  ↓ indexed by
Backend / Indexer
```

6. Why This Architecture Is Strong

This design demonstrates:

- dynamic on-chain authorization
- governance separation
- scalable append-only certificate issuance
- decentralized issuer management
- eUTXO state modeling
- Cardano-native architecture

rather than using a single centralized mutable database.

7. Recommended Design for the Project

Recommended approach:

- Registry UTXO → issuer authorization only
- Certificate minting policy → controls issuance
- Certificate tokens/NFTs → represent certificates
- IPFS → stores PDFs/images
- Certificate metadata/datum → stores hashes and references
- Indexer/backend → enables searching by gemId, issuer, reportId

This avoids using the registry as a large mutable storage structure while maintaining decentralized authorization and verifiable issuance.